

Here is the complete, detailed version history and future roadmap for **Project JARVIS**.

This follows the exact progression we locked into your **JARVIS Development Handbook (REDF v1.1)**. Every version builds on the previous one, and you cannot advance to the next version until the current one passes its specific functional tests.

Phase 1: Hardware & Foundation

v0.1 — The Assembly & Safety Release

- **The Goal:** Build the physical chassis, ensure electrical safety, and set up the software workspace. The robot doesn't move yet, but it stands on its own.
- **Key Milestones:**
 - Linux (Ubuntu) environment and Git repository initialized.
 - Robot is fully assembled using threadlocker/locknuts and balances passively on its feet.
 - Electrical safety checks: Fuses verified, battery polarity checked, and an emergency power disconnect is accessible.
 - Power isolation: The ESP32 logic board and the high-current servo motors are on separate power delivery paths.
- **Deliverables:** Wiring diagram documented, v0.1 Git Tag created, and Project Report pushed to GitHub.

Phase 2: Embedded Control

v0.2 — The Servo Control Release

- **The Goal:** Establish reliable software-to-hardware communication. You write the core C++ architecture (Robot → Leg → Servo) and calibrate the motors.
- **Key Milestones:**
 - You can command specific servos to move to exact angles using PlatformIO.
 - Mechanical zero, gear backlash, and soft limits are calibrated and saved in a configuration file so the robot doesn't strip its own gears.
 - Electrical stability: All 9 servos can move simultaneously without causing the ESP32 to brownout or reset.
 - Software timing: The control loop maintains a strict 50 Hz frequency without using blocking delay() functions.
- **Deliverables:** Video of controlled joint movement, calibration offsets saved, v0.2 Git Tag, Project Report.

Phase 3: Kinematics & Motion

v0.3 — The Walking Release

- **The Goal:** The robot takes its first steps. You apply Forward Kinematics (FK) and trigonometry to create a stable, hard-coded walking gait.
- **Key Milestones:**
 - Robot successfully walks 2 meters forward without tipping over (Center of Mass remains stable).
 - Robot can return cleanly from walking to a neutral standing pose.
 - **Repeatability Test:** The robot executes 10 consecutive flawless walking runs.
 - Metrics tracked: Average walking speed (cm/s) and battery runtime.
- **Deliverables:** Video proof of walking, v0.3 Git Tag, Project Report.

Phase 4: Basic Autonomy

v0.4 — The Obstacle Detection Release

- **The Goal:** Closing the loop (Sense → Decide → Act). JARVIS uses sensors to react to its environment using a Finite State Machine (IDLE → WALK → OBSTACLE_STOP).
- **Key Milestones:**
 - Ultrasonic sensor data is filtered (using median filters) to remove noise.
 - Robot detects obstacles in its path and smoothly decelerates to a stop.
 - Hysteresis is implemented so the robot doesn't "stutter" if an object is hovering exactly on the threshold boundary.
 - Metrics tracked: Stop latency (how fast it reacts) and false-positive rate.
- **Deliverables:** Video of autonomous stopping, v0.4 Git Tag, Project Report.

Phases 5 & 6: The Advanced Evolution (Future Roadmap)

Once v0.4 is complete, the embedded foundation is flawless. JARVIS graduates to advanced edge intelligence.

v0.5 — ROS 2 Integration

- The ESP32 is upgraded with **micro-ROS**.
- Instead of standard C++ loops, JARVIS operates using ROS 2 Nodes, Topics, and Publishers. It can subscribe to commands like `/cmd_vel` from an external computer.

v0.6 — Inverse Kinematics (IK)

- Static gait tables are replaced with dynamic math.
- Instead of commanding angles, you command the foot to move to a specific (X, Y, Z) coordinate, and the robot calculates the required servo angles on the fly using the Law of Cosines.

v0.7 — Machine Learning (Terrain Adaptation)

- Integrating your existing ML knowledge.
- JARVIS uses IMU telemetry and Softmax classification algorithms to "feel" what kind of surface it is walking on (e.g., carpet, tile, grass) and adjusts its step height automatically.

v0.8 — Path Planning

- Instead of just stopping when it sees an obstacle (like in v0.4), JARVIS maps the obstacle and calculates a route *around* it using algorithms like A* or Dijkstra.

v1.0 — The Fully Autonomous Quadruped

- Complete system integration. JARVIS is now a university-grade research platform ready for your Master's degree portfolio.